

Introduction to Computing (CS 101)

Introduction to Arrays

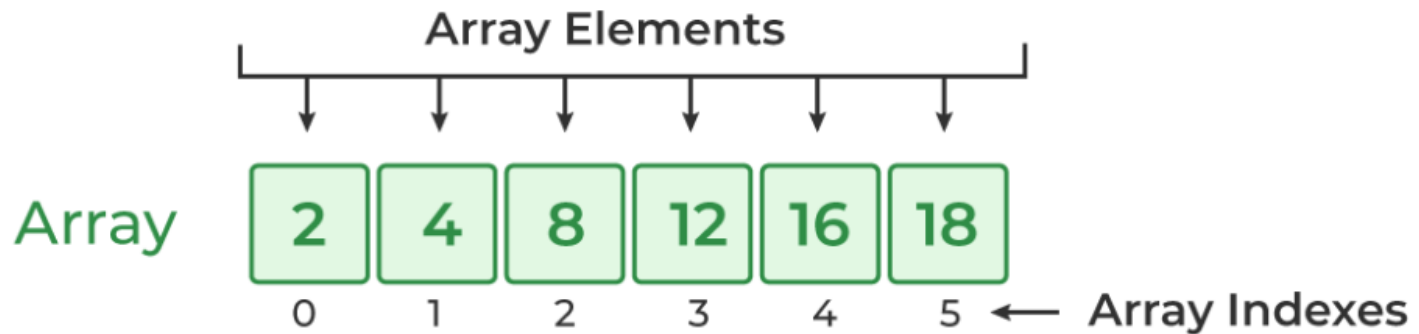
T. Venkatesh & John Jose



**Department of Computer Science & Engineering
Indian Institute of Technology Guwahati**

Definition – Array

- A collection of objects of the same type stored contiguously in memory under one name.
- Can be of any data type (int, char, float..)
- Provides ease of access to any member of array



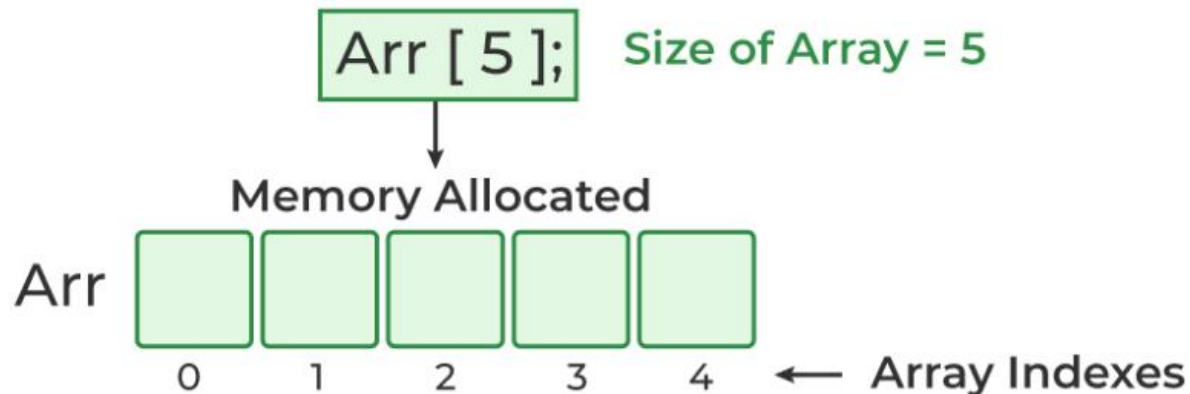
Array Declaration

- Declare an array by specifying its name, the type of its elements, and the size of its dimensions.
- When we declare an array the compiler allocates the memory block of the specified size to the array name.
- Syntax:

```
data_type array_name [size];
```

```
data_type array_name [size1][size2]..;
```

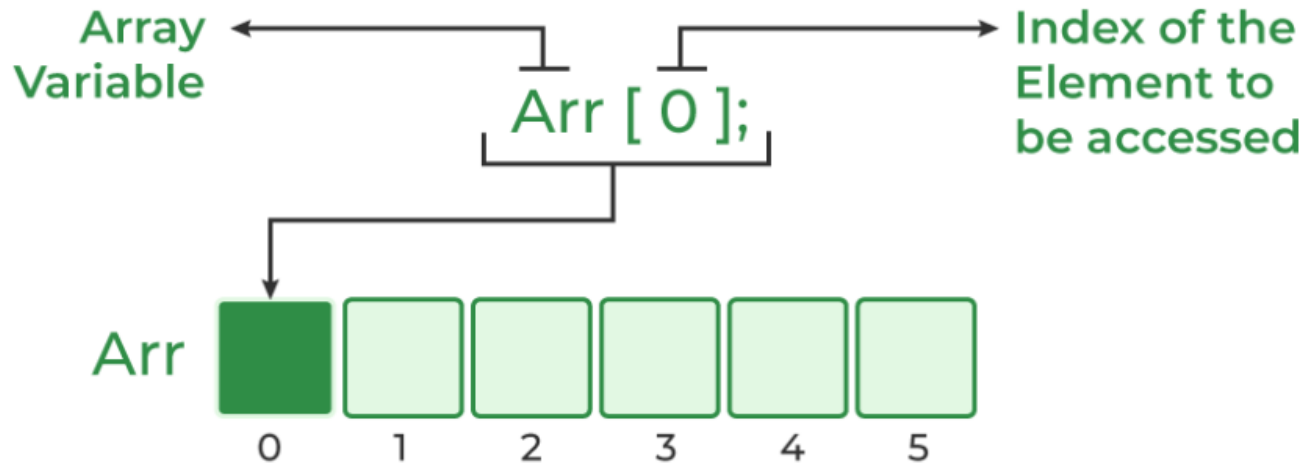
```
int Arr[5];
```



Array Declaration

- **Array indexes always start at zero in C**
- `int A[10]`
 - An array of ten integers
 - **`A[0], A[1], ..., A[9]`**
- `float B[20]`
 - An array of twenty long floating point numbers
 - **`B[0], B[1], ..., B[19]`**
- `int D[10][20]`
 - Two dimensional array
 - An array of ten rows, each of which is an array of twenty integers
 - **`D[0][0], D[0][1], ..., D[1][0], D[1][1], ..., D[9][19]`**

Use of Array Element



Usage of array in assignment and as a variable

```
A[3] = x + y;  
x = y - A[3];  
z = sin(A[i]) + cos(B[j]);
```

Arrays – Definition and Usage

```
int A[4];  
int b;  
A[0] = 3;  
A[3] = 4;
```

All elements of same type

Array size in declaration

First element (index 0)

Last element (index size - 1)

0x1014

0x1010

0x100C

0x1008

0x1004

0x1000

b

A[3]

A[2]

A[1]

A[0]

Arrays – Definition and Usage

- No bounds checking!
- Allowed – usually causes no obvious error
- A[4] may overwrite b

```
int A[4];
```

```
int b;
```

```
A[0] = 3;
```

```
A[3] = 4;
```

```
A[4] = 5;
```

```
A[-1] = 6;
```

0x1014

0x1010

b

0x100C

A[3]

0x1008

A[2]

0x1004

A[1]

0x1000

A[0]

Array Bound Checking

- It is the programmer's responsibility to avoid indexing off the end of an array
 - Likely to corrupt data
 - May cause a segmentation fault
 - Could expose system to a security hole!
- C does **NOT** check array bounds
 - No check whether index points to an element within the array
 - Might be high (beyond the end) or negative (before the array starts)

Array Initialization and Accessing

```
int A[5], i; //defining  
//initializing  
for (i=0; i<5; i++)  
    A[i]=i;  
//accessing the array  
for (i=0; i<5; i++)  
    printf("%d, ", A[i]);
```

Accessing Array Elements

- Array elements are commonly used in loops

```
for (i=0; i < max; i++)  
    A[i] = i*i;
```

```
sum = 0;  
for (j=0; j<max; j++)  
    sum += B[j];
```

```
sum = 0;  
for (count=0; count<30; count++) {  
    scanf("%f", &A[count]);  
    Sum += A[count];  
}
```

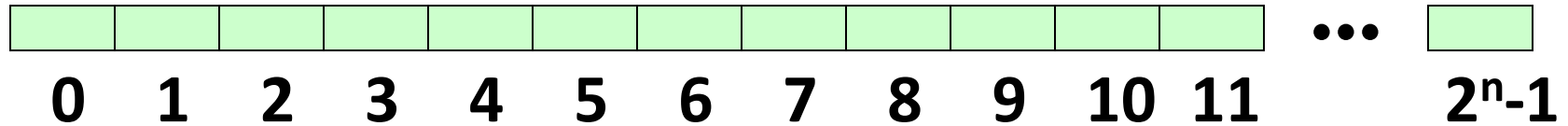
Array Sizes

```
int A[10];
```

- Size of object/data : returns the size of an object in bytes
- What is `sizeof(A[3])`? 4
- What is `sizeof(A)`? 40

```
int A[10];  
printf("%d, %d, %d ",  
        sizeof(int), sizeof(A[3]), sizeof(A));  
  
//outputs 4, 4, 40
```

Memory Organization Recap



- Memories are organized as sequence of numbered bytes
- Larger data types are sequences of bytes
 - **short int** – 2B
 - **int** – 4B
 - **long** – 8B
 - **float** – 4B
 - **double** – 8B
- Always aligned to multiple of size in bytes
- Address is the first byte of sequence
 - May be low-order or high-order byte
 - Big endian or Little endian

Array Representation

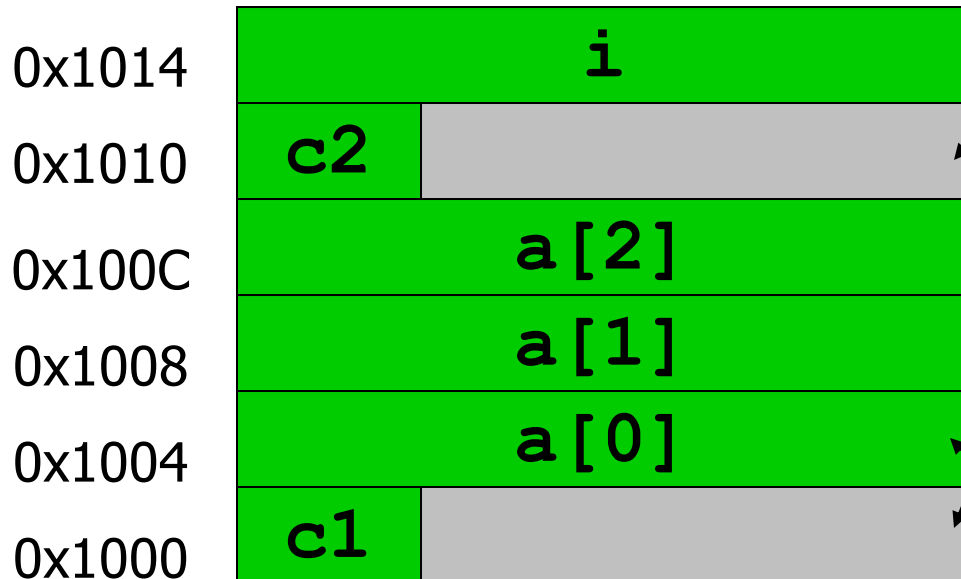
- Homogeneous: Each element same size –**s** bytes
 - An array of **m** data is a sequence of **m×s** bytes
 - Indexing: 0th data at byte **s×0**, 1st data at byte **s×1**, ...
- **m** and **s** are not part of representation
 - **s** known by compiler – usually irrelevant to programmer
 - **m** often known by compiler – if not, must be saved by programmer

0x1008	a[2]
0x1004	a[1]
0x1000	a[0]

```
int a[3];
```

Array Representation

```
char    c1;  
int     a[3];  
char    c2;  
int     i;
```



Could be optimized by making these adjacent, and reducing padding (by default, not)

Array aligned by size of elements

Array: Address, Element Address

- scanf require element address
- Reading values from keyboard and storing in array element

```
int A[10];  
int Sum = 0;  
for (count=0; count<10; count++) {  
    scanf("%f", &A[count]);  
    Sum += A[count];  
}
```

Array Initialization

- `int A[5] = {2, 4, 8, 16, 32};`
- `int B[20] = {2, 4, 8, 16, 32};`
 - Unspecified elements are guaranteed to be zero
- `int C[4] = {2, 4, 8, 16, 32};`
 - Error — compiler detects too many initial values

Computing Largest Element

```
int i, data[10], Largest;
printf("Enter 10 elements: ");
for(i=0; i < 10; ++i)
    scanf("%d", &data[i]);
Largest=data[0];
for(i=1; i<10; ++i) {
    if(Largest<data[i])
        Largest=data[i];
}
printf("Largest Element=%d\n",Largest);
```

Computing Average

```
main()
{
    int i, n, sum=0;
    int age[50];
    scanf("%d", &n); //n should be less than 50
    for(i=0 ; i< n ; i++) {
        printf("Age of next person: ");
        /* accessing array elements */
        scanf("%d", &age[i]);
        sum += age[i];
    }
    printf("The ages of persons are:\n");
    for(i=0 ; i<n ; i++)
        printf("%d\n", age[i]);
    printf("The average is: %f\n", sum / n);
}
```

Find largest and smallest element

```
main()
{
    int    i, n;
    float a[50], large, small;

    printf("Size of Array? ");
    scanf("%d", &n);
    printf("\n Array Elements? :\n");

    for (i=0 ; i<n ; i++)
        scanf("%f", &a[i]);
```

Find largest and smallest element

```
/* Initialisation */
large = a[0];
small = a[0];

/* largest & smallest element */
for (i=1; i<n ; i++) {
    if (a[i] > large)
        large = a[i];
    else if (a[i] < small)
        small = a[i];
}
printf("\n Largest: %f\n", large);
printf("\nSmallest: %f\n", small);
}
```

Computing Standard deviation

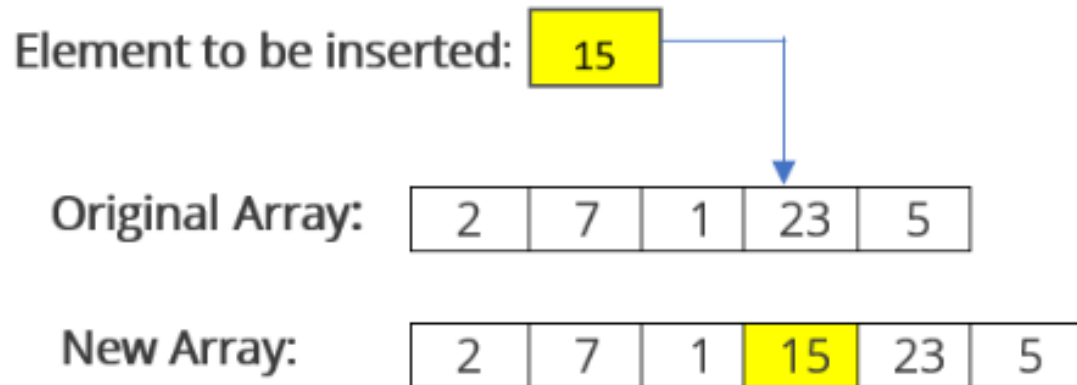
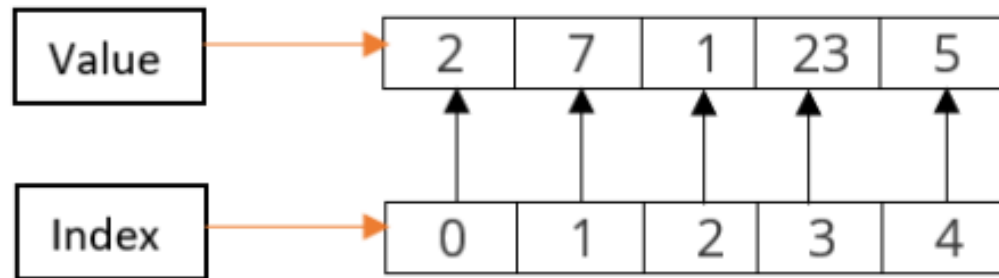
$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

```
int i;
float data[10], sum=0.0, mean, SD=0.0;
printf("Enter 10 elements: ");
for(i=0; i < 10; ++i)
    scanf("%f", &data[i]);

for(i=0; i<10; ++i) sum += data[i];
mean = sum/10;
for(i=0; i<10; ++i)
    SD += (data[i]-mean) * (data[i]-mean);
SD = sqrt(SD/10);
printf("\nStandard Deviation=%f", SD);
```

Insert an element into an Array

Example: `arr[5] = {2,7,1,23,5}`



Insert an element into an Array

```
main()  
{  
    int    i, j, k, n, pos;  
    int    a[50], item;  
  
    printf("Size of array? ");  
    scanf("%d", &n);  
    printf("\n Array Elements?:\n");  
    for (i=0 ; i<n ; i++)  
        scanf("%d", &a[i]);  
  
    printf("\n Element to be inserted?: ");  
    scanf("%d", &item);  
    printf("\n Position of insertion?: ");  
    scanf("%d", &pos);
```

Insert an element into an Array

```
/* pushing down elements */
for (k=n ; k >= pos ; k--)
    a[k] = a[k-1];
/* insert item */
a[--pos] = item;

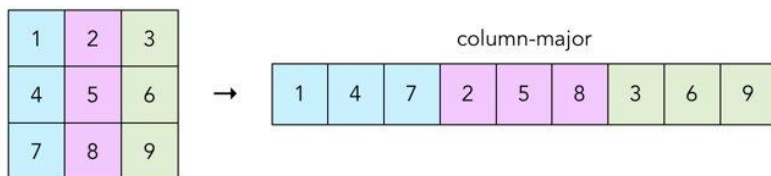
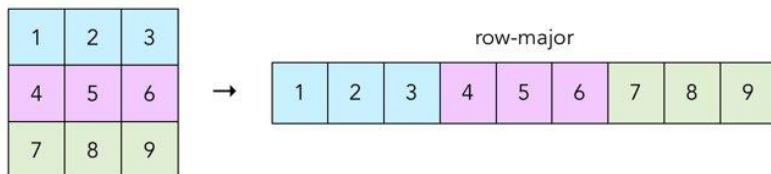
printf("\nArray after insertion:\n");
for (i=0 ; i<(n+1) ; i++)
    printf("%d", a[i]);

printf("\n");
}
```


Multidimensional Arrays

- Useful to store two/three/... dimensional data structures
 - Example: Matrix
 - Array will have two subscripts
 - one for row and one for column
- `float m[10][20];`
 - 10 rows, 20 columns
 - First row, first column: `m[0][0]`
 - Last row, last column: `m[9][19]`
- Memory storage: first row - all columns, then second row - all columns, ...
- `m[0][0]`, `m[0][1]`, ... `m[0][19]`, then
- `m[1][0]`, `m[1][1]`, ... `m[1][19]`, then `m[2][0]`...

Multi-Dimensional Arrays



C follows row major format

A: array[0..3,0..3] of char

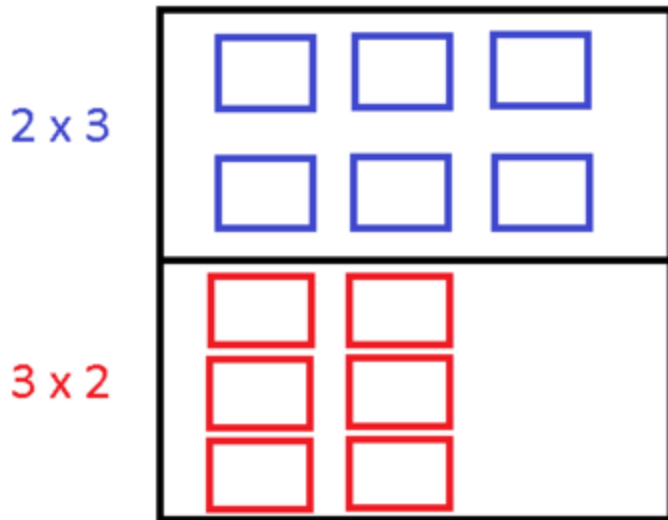
	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

Memory

	15	A[3,3]
	14	A[3,2]
	13	A[3,1]
	12	A[3,0]
	11	A[2,3]
	10	A[2,2]
	9	A[2,1]
	8	A[2,0]
	7	A[1,3]
	6	A[1,2]
	5	A[1,1]
	4	A[1,0]
	3	A[0,3]
	2	A[0,2]
	1	A[0,1]
	0	A[0,0]

Multi-Dimensional Arrays

Row Major Organization



```
int  matrix[2][3];
```

```
matrix[1][0] = 17;
```

0x1014

matrix[1][2]

0x1010

matrix[1][1]

0x100C

matrix[1][0]

0x1008

matrix[0][2]

0x1004

matrix[0][1]

0x1000

matrix[0][0]

What happens when you write: **matrix[0][3] = 42;**

2D array Initialization and access

```
int i, j;
int A[2][5] = {{20, 11, 22, 33, 44},
               {1, 12, 23, 34, 25}};
for (i=0; i<2; i++) {
    for (j=0; j<5; j++) {
        printf("%d, ", A[i][j]);
    }
}
//prints 20, 11, 22, 33, 44, 1, 12, 23, 34,
25,
```

Implicit Array Size Determination

```
int one_d[]={31,28,31,30,31,30,31,31,30,31};
```

- Array is created with as many elements as initial values
- In this case, 12 elements
- Values must be compile-time constants (for static arrays)

```
int two_d[3][4] = {{1, 2, 3, 4}, {2, 3, 4, 5},  
{5, 6, 7, 8}};
```

- Can be written as: [no first subscript]

```
int two_d[ ][4] = { {1, 2, 3, 4}, {2, 3, 4,  
5}, {5, 6, 7, 8}};
```

- Can we keep the first subscript and omit the second ?
- Can be written as: [no inner braces]

```
int two_d[ ][4] = { 1, 2, 3, 4, 2, 3, 4, 5, 5,  
6, 7, 8 };
```

Matrix Addition

$$\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 3 & 2 \\ 1 & 4 \\ 5 & 3 \end{bmatrix}$$

✗ We cannot add them

But we can add

$$\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} -9 & 2 \\ -1 & 3 \end{bmatrix}$$

✓ We can add them

$$\begin{bmatrix} 3 & 2 \\ 1 & 4 \\ 5 & 3 \end{bmatrix} \begin{bmatrix} -2 & 5 \\ -8 & 12 \\ 0 & -18 \end{bmatrix}$$

✓ We can add them

Program for addition of two matrices

$$\begin{bmatrix} 0 & 4 \\ 1 & 3 \end{bmatrix} + \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 0+3 & 4+2 \\ 1+5 & 3+1 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & 6 \\ 6 & 4 \end{bmatrix}$$

Matrix Addition

```
// adds A[][] and B[][] and stores in C[][]
int main()
{
    int i, j;
    int A[4][4] = { { 1, 1, 1, 1 },
                    { 2, 2, 2, 2 },
                    { 3, 3, 3, 3 },
                    { 4, 4, 4, 4 } };

    int B[4][4] = { { 2, 2, 2, 2 },
                    { 3, 3, 3, 3 },
                    { 4, 4, 4, 4 },
                    { 5, 5, 5, 5 } };
```

Matrix Addition

```
// adds A[][] and B[][] and stores in C[][]  
for (i = 0; i < 4; i++) {  
    for (j = 0; j < 4; j++)  
        C[i][j] = A[i][j] + B[i][j];  
}  
  
for (i = 0; i < 4; i++) {  
    for (j = 0; j < 4; j++)  
        printf("%d ", C[i][j]);  
    printf("\n");  
}  
}
```


Thanks